

Version Control with Git

1. Team-Based Development

Modern software is developed collaboratively. Multiple developers work on the same codebase, often in parallel. Therefore, a development environment must support:

- Shared access to code
- Independent development without overwriting
- Safe merging of changes
- Traceability of who changed what and when
- Reversible history
- Backup
- Conflict resolution

This capability is enabled by **Version Control Systems (VCS)**.

2. Issues in Collaborative Development Without Version Control

If teams share files manually (email, zip files, copying folders, etc.), the following issues arise:

- Files get **overwritten accidentally**
- **No record/history** of changes
- Cannot **restore previous versions**
- Two developers editing the same file causes **conflicts**
- No way to track **who changed what**
- Debugging becomes complex
- No **backup**
- Hard to manage dependencies



Conclusion: A structured **Version Control System (VCS)** is mandatory.

3. Version Control Systems — Concepts & Types

Definition

A Version Control System tracks, manages, and stores changes to software over time.

Types of VCS

Type	Description	Examples
Local VCS	Versions stored on one computer	RCS
Centralized VCS	Single shared repository	CVS, Subversion (SVN)
Distributed VCS	Every developer has a full copy	Git, Mercurial

4. Why Git? (Industry Standard)

Git is the dominant VCS because it is:

- Distributed
- Fast
- Secure
- Reliable
- Supports offline work
- Enables powerful branching & merging
- Open-source and cross-platform

Every developer has a **full local repository**, reducing dependency on the server.

5. Git Repository & Internal Structure

A **repository (repo)** is the database storing project history.

Areas in Git

Area	Meaning
Working Directory	Files currently being worked on
Staging Area (Index)	Files prepared for commit
Local Repository	Permanent commit history
Remote Repository	Shared online repo (GitHub, GitLab, Bitbucket)

Workflow

Working Directory → Staging → Local Repo → Remote Repo

6. Git Repository Creation & Setup

Initialize a repository

```
git init
```

Clone existing repository

```
git clone <url>
```

7. Adding & Committing Code

Check status

```
git status
```

Stage files

```
git add filename
```

```
git add .
```

Commit snapshot

`git commit -m "meaningful message"`

A **commit** = **version snapshot**.

View commit history

`git log`

8. Updating & Synchronizing Code

Pull latest changes

`git pull`

Push local changes

`git push`

9. Branching — Core of Git

What is a branch?

A branch is a parallel line of development enabling multiple features simultaneously.

Create branch

`git branch feature1`

Switch branch

`git checkout feature1`

Create & switch

`git checkout -b feature1`

10. Merging — Integrating Changes

Merge feature into main

`git checkout main`

`git merge feature1`

Delete branch

`git branch -d feature1`

11. Fetching & Comparing Changes

Fetch without merging

`git fetch`

View differences

git diff

12. Handling Git Conflicts

Conflicts occur when two developers modify the same code block.

Git **marks conflict areas** and the developer must resolve manually, then recommit.

13. Industry-Standard Git Workflow

1. Create feature branch
2. Develop code
3. Stage & commit
4. Push branch
5. Submit Pull Request (PR)
6. Code review
7. Merge to main branch
8. Deploy

This ensures **traceability, quality, and safe collaboration**.

14. Software Development Workflow with Git

Git supports the complete flow:

- Coding
- Review
- Build
- Test
- Deploy

and enables:

- ✓ Continuous Integration
 - ✓ Continuous Delivery
 - ✓ DevOps automation
-